

Main is chapter ka central idea ye hai: **JavaScript mein code reuse ke liye class aur inheritance compulsory nahi hain. Objects, [[Prototype]], aur dynamic this ko combine karke objects ek-doesre ko kaam delegate kar sakte hain.** Is approach ko chapter **delegation, virtual composition**, aur kuch contexts mein **OLOO** kehta hai.

Important: Chapter abhi “work in progress” hai. Isliye text mein kuch typos aur ek technical wording mistake bhi hai, jise main relevant jagah par correct karunga.

1. Delegation ka basic meaning

Delegation ka simple meaning hai:

Ek object har kaam khud na karke, kisi doosre object se woh kaam karwaye.

For example:

- Coordinates coordinate manage kare
- Canvas drawing manage kare
- ControlPoint ek actual point ko represent kare
- Ye objects inheritance ke parent-child relation mein hone zaroori nahi
- Phir bhi ye this, .call() aur prototype linkage ke through cooperate kar sakte hain

Traditional class design mein hum usually sochte hain:

```
1 Parent class
2   ↓
3 Child class
4   ↓
5 Instance
```

Delegation design mein thinking kuch aisi hai:

```
1 Coordinates ↔ ControlPoint ↔ Canvas
```

Yahaan ye objects ek-doesre ke **peers/cooperators** hain, na ki strictly parent aur child.

2. Constructor actually object create nahi karta

Class mein hum generally likhte hain:

```
1 class Point2d {
2   constructor(x, y) {
3     this.x = x;
```

```
4         this.y = y;
5     }
6 }
7
8 const point = new Point2d(3, 4);
```

Aisa lag sakta hai ki constructor() object create kar raha hai. Lekin actually:

- new object create karta hai
- new prototype linkage set karta hai
- uske baad constructor ko call karta hai
- constructor newly created object ko initialize karta hai

Isliye constructor ka actual kaam hai:

```
1 Creation nahi
2 Initialization
```

constructor() chalne tak instance pehle hi create ho chuka hota hai.

3. Factory function se object banana

Chapter pehle ek simple factory function dikhata hai:

```
1 function Point2d(x, y) {
2     var instance = {};
3
4     instance.x = x;
5     instance.y = y;
6
7     return instance;
8 }
9
10 var point = Point2d(3, 4);
```

Isme:

```
1 var instance = {};
```

Naya object create karta hai.

```
1 instance.x = x;
2 instance.y = y;
```

Object initialize karta hai.

```
1 return instance;
```

Created object caller ko return karta hai.

Is pattern ko **factory function** kehte hain.

Factory function kya hota hai?

Factory function ek normal function hota hai jo:

1. naya object create karta hai
2. us object ko initialize karta hai
3. object return karta hai

Iske saath new lagana necessary nahi hota:

```
1 const point = Point2d(3, 4);
```

4. new ke four conceptual steps

Chapter ke according new ke kaam ko roughly four steps mein samjha ja sakta hai:

1. Naya empty object create karna
2. Us object ka [[Prototype]] link set karna
3. Function/constructor ko naye object ke this context mein call karna
4. Object return karna

Conceptually ye kuch aisa lag sakta hai:

```
1 function conceptualNew(Constructor, ...args) {  
2     const instance = Object.create(Constructor.prototype);  
3  
4     Constructor.apply(instance, args);  
5  
6     return instance;  
7 }
```

Actual new ke return-value rules thode aur nuanced hote hain, lekin chapter ke current topic ke liye ye useful mental model hai.

5. Factory object ko prototype se link karna

Initial factory mein object create aur initialize to ho raha tha, lekin prototype delegation nahi thi.

Chapter isko modify karta hai:

```

1  var prototypeObj = {
2      toString() {
3          return `(${this.x},${this.y})`;
4      },
5  };
6
7  function Point2d(x, y) {
8      var instance = {
9          __proto__: prototypeObj,
10     };
11
12     instance.x = x;
13     instance.y = y;
14
15     return instance;
16 }

```

Ab instance ke paas apna toString() method directly nahi hai.

Jab hum likhte hain:

```

1  point.toString();

```

JavaScript resolution roughly aise karta hai:

1. point ke andar toString search karo
2. nahi mila to point.[[Prototype]] par jao
3. prototypeObj.toString mil gaya
4. method ko call karo

Lekin sabse important baat:

```

1  point.toString();

```

mein function prototypeObj par mila, lekin this phir bhi point hoga.

Isliye:

```

1  return `(${this.x},${this.y})`;

```

mein this.x aur this.y, point.x aur point.y ko read karenge.

Important distinction

- | | |
|---------------------------|------------------|
| 1 Method kahaan mila? | prototypeObj par |
| 2 Method ka this kya hai? | point |

Yahi prototype delegation ka main mechanism hai.

6. __proto__ versus Object.create()

Chapter illustration ke liye ye use karta hai:

```
1 var instance = {  
2   __proto__: prototypeObj,  
3 };
```

Lekin cleaner/direct approach hai:

```
1 var instance = Object.create(prototypeObj);
```

Dono ka intended effect:

```
1 instance.__proto__ → prototypeObj
```

Alternative:

```
1 var instance = {};  
2 Object.setPrototypeOf(instance, prototypeObj);
```

Chapter Object.create() ko prefer karta hua pattern derive karta hai, kyunki ye creation aur linkage ek hi operation mein kar deta hai.

7. Factory function ke saath new use karne ki problem

Consider:

```
1 function Point2d(x, y) {  
2   var instance = Object.create(prototypeObj);  
3  
4   instance.x = x;  
5   instance.y = y;  
6  
7   return instance;  
8 }  
9  
10 var point = new Point2d(5, 6);
```

Yahaan strange situation hoti hai.

new kya karega?

new pehle apna object create karega:

```
1 object A
```

Phir Point2d() function chalega. Lekin function khud ek aur object create kar raha hai:

```
1 object B
```

Aur function explicitly object B return kar deta hai.

Normal constructor invocation mein, agar function explicitly koi object return kare, to new ka result woh explicitly returned object ban sakta hai. Is wajah se new ka banaya hua object A use nahi hota.

Conceptually:

```
1 new ne object A banaya
2 Point2d ne object B banaya
3 Point2d ne object B return kiya
4 final result = object B
5 object A = unused
```

Isliye factory function ko new ke saath call karna unnecessary aur conceptually wasteful hai.

Code mein ek small issue

Chapter mein diya hai:

```
1 anotherPoint.toString(5, 6);
```

Lekin toString() arguments accept nahi kar raha:

```
1 toString() {
2     return `${this.x},${this.y}`;
3 }
```

Extra arguments JavaScript mein ignore ho jayenge. Clearer call hota:

```
1 anotherPoint.toString(); // "(5,6)"
```

8. Initialization ko init() mein shift karna

Chapter initialization ko factory function se nikaal kar prototype/delegate object par rakhta hai:

```
1 var prototypeObj = {
2     init(x, y) {
3         this.x = x;
4         this.y = y;
5     },
6
7     toString() {
8         return `${this.x},${this.y}`;
9     },
}
```

```

10 };
11
12 function Point2d(x, y) {
13     var instance = Object.create(prototypeObj);
14
15     instance.init(x, y);
16
17     return instance;
18 }

```

Yahaan:

```

1 instance.init(x, y);

```

mein init() method prototypeObj par milega, lekin implicit method call ki wajah se:

```

1 this === instance

```

Isliye:

```

1 this.x = x;
2 this.y = y;

```

properties prototypeObj par nahi, instance par create hongi.

Result:

```

1 prototypeObj
2   └─ init()
3   └─ toString()
4
5 instance
6   └─ x
7   └─ y
8   └─ [[Prototype]] → prototypeObj

```

Methods shared hain, data har instance ka apna hai.

9. Function ko hata kar sirf object rakhna

Ab chapter ek important switch karta hai:

```

1 var Point2d = {
2     init(x, y) {
3         this.x = x;
4         this.y = y;
5     },

```

```

6
7     toString() {
8         return `${this.x},${this.y}`;
9     },
10 };

```

Ab Point2d:

- class nahi hai
- constructor function nahi hai
- ek normal object hai
- shared behavior contain karta hai

Instance creation:

```

1  var point = Object.create(Point2d);
2  point.init(3, 4);
3
4  point.toString(); // "(3,4)"

```

Is structure ko aise visualize karo:

```

1  point
2  └─ x: 3
3  └─ y: 4
4  └─ [[Prototype]] ───┐
                        ↓
5                        Point2d
6                        └─ init()
7                        └─ toString()
8

```

Jab:

```

1  point.toString();

```

call hota hai:

- method Point2d par milta hai
- this point hota hai
- output point ke coordinates se banta hai

10. Object.create() ke description mein correction

Chapter ke text mein likha hai ki Object.create():

new empty object ko function ke .prototype object se link karta hai.

Yeh wording is context mein technically incorrect hai.

Actual behavior:

```
1 Object.create(someObject)
```

directly new object ka `[[Prototype]]`, **passed object** ko banata hai:

```
1 newObject.[[Prototype]] → someObject
```

Example:

```
1 const point = Object.create(Point2d);
```

Result:

```
1 point.[[Prototype]] → Point2d
```

Ye nahi hota:

```
1 point.[[Prototype]] → Point2d.prototype
```

Yahaan Point2d ek ordinary object hai, function nahi. Isliye `.prototype` ki zarurat hi nahi.

new se comparison

`new SomeFunction()` generally linkage banata hai:

```
1 instance.[[Prototype]] → SomeFunction.prototype
```

Lekin:

```
1 Object.create(someObject)
```

linkage banata hai:

```
1 instance.[[Prototype]] → someObject
```

Ye difference bahut important hai.

11. Creation aur initialization separate hain

Delegation style mein:

```
1 var point = Object.create(Point2d);
2 point.init(3, 4);
```

Do operations separate hain:

Creation and linkage

```
1 var point = Object.create(Point2d);
```

Initialization

```
1 point.init(3, 4);
```

Class style mein ye dono ek expression mein ho jaate hain:

```
1 var point = new Point2d(3, 4);
```

Delegation style ka cost ye hai ki creation aur initialization ke liye do statements likhne pad rahe hain.

Lekin benefit ye hai ki mechanics explicit dikh rahi hain.

12. Generic make() helper

Agar do statements inconvenient lagti hain, chapter ek factory helper banata hai:

```
1 function make(objType, ...args) {  
2     var instance = Object.create(objType);  
3     instance.init(...args);  
4     return instance;  
5 }  
6  
7 var point = make(Point2d, 3, 4);
```

make() ke steps:

```
1 var instance = Object.create(objType);
```

New object create karta hai aur objType se prototype-link karta hai.

```
1 instance.init(...args);
```

objType se delegated init() ko instance ke context mein call karta hai.

```
1 return instance;
```

Initialized object return karta hai.

Multiple objects:

```
1 var point = make(Point2d, 3, 4);  
2 var anotherPoint = make(Point2d, 5, 6);
```

Dono ka data separate hoga:

```
1 point.x = 3  
2 point.y = 4  
3  
4 anotherPoint.x = 5  
5 anotherPoint.y = 6
```

Lekin methods shared honge:

```
1 point.[[Prototype]] —————
```

```
2          |-----> Point2d
3  anotherPoint.[[Prototype]] -----|
```

Implied convention

make() assume karta hai ki har delegated object ke paas init() naam ka method hoga:

```
1  instance.init(...args);
```

Agar objType ya uski prototype chain par init() nahi hua, to error aayega.

13. Class thinking ko abandon karna

Chapter ka next step sirf class syntax hatana nahi hai. Author class-oriented **thinking** bhi hatana chahta hai.

Class-oriented thinking

Class design mein hum categories aur hierarchy banate hain:

```
1  Shape
2    ↓
3  Point
4    ↓
5  ControlPoint
```

Yahaan:

- base class generalized hoti hai
- subclass specialized hoti hai
- instance class ka concrete version hota hai
- behavior inheritance chain ke through vertically compose hota hai

Isse vertical composition keh sakte hain:

```
1  Base
2    ↓
3  Derived
4    ↓
5  MoreDerived
```

Delegation thinking

Delegation mein hum puchte hain:

Kaunse concrete objects ko kaunsa kaam karna aata hai, aur ye objects runtime par kaise cooperate kar sakte hain?

Yahaan focus classification par nahi, **collaboration** par hota hai.

- | | | |
|---|--------------------|------------------------------------|
| 1 | Coordinates object | → coordinate operations |
| 2 | Inspect object | → inspection/string representation |
| 3 | Canvas object | → drawing operations |
| 4 | ControlPoint | → actual control-point behavior |

Ye objects ek hierarchy banane ke bajaye runtime par collaborate karte hain.

14. Explicit delegation

Chapter ka example:

```
1  var Coordinates = {
2      setX(x) {
3          this.x = x;
4      },
5
6      setY(y) {
7          this.y = y;
8      },
9
10     setXY(x, y) {
11         this.setX(x);
12         this.setY(y);
13     },
14 };
15
16 var Inspect = {
17     toString() {
18         return `(${this.x},${this.y})`;
19     },
20 };
21
22 var point = {};
```

Ab:

1 Coordinates.setXY.call(point, 3, 4);
--

.call() first argument ko function ka this banata hai.

Therefore:

```
1 Coordinates.setXY.call(point, 3, 4);
```

ke andar:

```
1 this === point
```

To:

```
1 this.setX(x);
```

actually resolve karne ki koshish karega:

```
1 point.setX(x);
```

Lekin yahaan ek important issue hai

As written:

```
1 var point = {};  
2 Coordinates.setXY.call(point, 3, 4);
```

point ke paas setX() aur setY() methods nahi hain. Aur point ka prototype bhi Coordinates nahi hai.

Isliye Coordinates.setXY() ke andar:

```
1 this.setX(x);
```

jab this === point, to point.setX undefined hoga, aur call fail hoga.

Likely error:

```
1 TypeError: this.setX is not a function
```

Example ko working banane ke possible approaches

Agar explicit delegation hi dikhani hai, to setXY() ke andar direct explicit calls ki ja sakti hain:

```
1 var Coordinates = {  
2   setX(x) {  
3     this.x = x;  
4   },  
5  
6   setY(y) {  
7     this.y = y;  
8   },  
9  
10  setXY(x, y) {  
11    Coordinates.setX.call(this, x);  
12    Coordinates.setY.call(this, y);  
13  },  
14 };
```

```
15
16 var point = {};
17
18 Coordinates.setXY.call(point, 3, 4);
```

Ab point ke paas methods hona necessary nahi.

Ya point ko Coordinates se link kar sakte hain:

```
1 var point = Object.create(Coordinates);
2 point.setXY(3, 4);
```

Lekin ye phir implicit delegation ban jaayegi.

Chapter ka intended concept valid hai, lekin first explicit-delegation snippet as written problematic hai.

15. Inspect.toString.call(point)

Consider:

```
1 Inspect.toString.call(point);
```

Yahaan:

- function Inspect.toString se liya gaya
- this explicitly point banaya gaya
- function point.x aur point.y access karta hai

Conceptually:

```
1 function toString() {
2     return `${point.x},${point.y}`;
3 }
```

Lekin original function reusable hai kyunki usmein fixed point reference nahi, dynamic this hai:

```
1 toString() {
2     return `${this.x},${this.y}`;
3 }
```

Isi function ko kisi doosre compatible object ke saath bhi use kar sakte ho:

```
1 Inspect.toString.call(anotherPoint);
```

Ye **explicit delegation** hai, kyunki hum manually .call() se context select kar rahe hain.

16. Implicit delegation

Ab:

```
1 var anotherPoint = Object.create(Coordinates);  
2  
3 anotherPoint.setXY(5, 6);
```

anotherPoint par setXY() directly nahi hai. JavaScript uske prototype Coordinates par method find karta hai.

Lekin call-site:

```
1 anotherPoint.setXY(5, 6);
```

hone ki wajah se:

```
1 this === anotherPoint
```

Isko chapter **implicit delegation** kehta hai.

Implicit isliye, kyunki .call(anotherPoint, ...) explicitly nahi likha. Method call syntax ne automatically context select kiya.

Difference

Explicit delegation

```
1 Coordinates.setX.call(point, 3);
```

Humne explicitly kaha:

```
1 Coordinates ka method lo  
2 point ko this banao
```

Implicit delegation

```
1 anotherPoint.setX(3);
```

Prototype chain method find karti hai aur call-site automatically:

```
1 this = anotherPoint
```

set karta hai.

17. Virtual composition kya hai?

Traditional composition mein behaviors ko pehle combine kiya ja sakta hai:

```
1 class Point {  
2   setXY() {}
```

```
3     toString() {}
4 }
```

Ya mixin ke through methods ek object mein copy kiye ja sakte hain.

Delegation style mein behaviors physically ek object mein copy nahi hote. Runtime par this sharing se different objects ke behaviors ek context par work karte hain.

Example:

```
1 Coordinates.setX.call(point, 3);
2 Coordinates.setY.call(point, 4);
3 Inspect.toString.call(point);
```

Yahaan:

- setX() Coordinates se
- setY() Coordinates se
- toString() Inspect se
- actual data point par

Behaviors alag objects par stored hain, lekin sab same point context par work kar rahe hain.

Chapter is runtime combination ko **virtual composition** kehta hai.

```
1 Coordinates behavior ————
2                               } runtime this = point
3 Inspect behavior  ————
```

Functions physically point ke andar copy nahi hue. Lekin runtime par aisa effect milta hai jaise behaviors compose ho gaye hon.

18. OLOO ka meaning

OLOO ka full form hai:

Objects Linked to Other Objects

Is thinking mein:

- classes primary abstraction nahi
- objects primary abstraction hain
- objects prototype linkage ya this sharing se cooperate karte hain

Example:

```
1 var Point2d = {
2     init(x, y) {
```

```

3         this.x = x;
4         this.y = y;
5     },
6
7     toString() {
8         return `(${this.x},${this.y})`;
9     },
10 };
11
12 var point = Object.create(Point2d);
13 point.init(3, 4);

```

Yahaan:

```
1 point object → linked to → Point2d object
```

Koi classical class-instance requirement nahi hai. Bas objects hain, jo other objects se linked hain.

19. Canvas, Coordinates, aur ControlPoint

Ab chapter delegation ka complex example dikhata hai.

Teen main objects hain:

```

1 Canvas
2 Coordinates
3 ControlPoint

```

Inke responsibilities:

Canvas

```

1 var Canvas = {
2     setOrigin(x, y) {},
3     pixel(x, y) {},
4     renderScene() {},
5 };

```

Canvas drawing operations manage karta hai:

- coordinate origin move karna
- vertical axis flip karna
- pixel draw karna
- canvas clear karke scene redraw karna

Coordinates

```
1 var Coordinates = {  
2     setX(x) {},  
3     setY(y) {},  
4     setXY(x, y) {},  
5 };
```

Coordinate-related operations manage karta hai:

- x set karna
- y set karna
- x and y together set karna

ControlPoint

```
1 var ControlPoint = {  
2     __proto__: Coordinates,  
3  
4     ctx: /* canvas context */,  
5  
6     rotate(angleRadians) {},  
7     draw() {},  
8     render() {},  
9 };
```

ControlPoint:

- Coordinates se implicitly delegate karta hai
 - canvas context hold karta hai
 - rotation calculate karta hai
 - khud ko draw/render karta hai
-

20. ControlPoint ka prototype linkage

```
1 var ControlPoint = {  
2     __proto__: Coordinates,  
3 };
```

Iska meaning:

```
1 ControlPoint.[[Prototype]] → Coordinates
```

Agar hum call karein:

```
1 ControlPoint.setXY(30, 40);
```

To:

1. setXY ControlPoint par search hoga
2. wahan nahi milega
3. Coordinates par milega
4. method invoke hoga
5. this ControlPoint hi rahega

Again:

```
1 Method found on: Coordinates
2 this value:      ControlPoint
```

Prototype lookup this ko us object mein change nahi karta jahaan method mila.

21. Canvas origin set karna through explicit delegation

```
1 Canvas.setOrigin.call(ControlPoint, 100, 100);
```

Function Canvas.setOrigin hai:

```
1 setOrigin(x, y) {
2   this.ctx.translate(x, y);
3   this.ctx.scale(1, -1);
4 }
```

.call(ControlPoint, 100, 100) ki wajah se:

```
1 this === ControlPoint
```

Therefore:

```
1 this.ctx
```

actually:

```
1 ControlPoint.ctx
```

hai.

Canvas khud necessarily canvas context hold nahi kar raha. Canvas.setOrigin() ko context ControlPoint provide kar raha hai.

Ye explicit delegation hai:

```
1 Behavior Canvas se
2 State/context ControlPoint se
```

22. setXY() se render() kaise milta hai?

Code:

```
1  var Coordinates = {  
2      setXY(x, y) {  
3          this.setX(x);  
4          this.setY(y);  
5          this.render();  
6      },  
7  };  

```

Call:

```
1  ControlPoint.setXY(30, 40);  

```

setXY() Coordinates par mila, lekin:

```
1  this === ControlPoint  

```

Therefore:

```
1  this.setX(x);  

```

ControlPoint se lookup start karta hai:

1. ControlPoint.setX nahi hai
2. prototype Coordinates.setX milta hai
3. this still ControlPoint

Similarly:

```
1  this.render();  

```

lookup ControlPoint se start hoga. Aur render() directly ControlPoint par defined hai:

```
1  render() {  
2      Canvas.renderScene.call(this);  
3  }  

```

To Coordinates.setXY() indirectly ControlPoint.render() call kar raha hai.

Lekin Coordinates ne hard-code nahi kiya:

```
1  ControlPoint.render();  

```

Usne dynamic call kiya:

```
1  this.render();  

```

Isi wajah se Coordinates.setXY() kisi aur compatible object ke saath bhi use ho sakta hai.

23. Exact execution flow

Jab ye call hota hai:

```
1 ControlPoint.setXY(30, 40);
```

To complete flow hai:

```
1 ControlPoint.setXY(30,40)
2     ↓ prototype lookup
3 Coordinates.setXY(30,40)
4     ↓ this = ControlPoint
5 this.setX(30)
6     ↓
7 Coordinates.setX with this = ControlPoint
8     ↓
9 ControlPoint.x = 30
10    ↓
11 this.setY(40)
12    ↓
13 Coordinates.setY with this = ControlPoint
14    ↓
15 ControlPoint.y = 40
16    ↓
17 this.render()
18    ↓
19 ControlPoint.render()
20    ↓
21 Canvas.renderScene.call(ControlPoint)
22    ↓
23 Canvas.renderScene with this = ControlPoint
24    ↓
25 this.draw()
26    ↓
27 ControlPoint.draw()
28    ↓
29 Canvas.pixel.call(ControlPoint,30,40)
30    ↓
31 ControlPoint.ctx.fillRect(30,40,1,1)
```

Ye poora flow delegation ke heart ko demonstrate karta hai.

24. Canvas.renderScene() ko draw() kahaan se mila?

Code:

```
1 renderScene() {  
2     // canvas clear...  
3     this.draw();  
4 }
```

Canvas object par draw() method defined nahi hai.

Phir this.draw() kaise work karta hai?

Because call ye hai:

```
1 Canvas.renderScene.call(ControlPoint);
```

So:

```
1 this === ControlPoint
```

Isliye:

```
1 this.draw();
```

means:

```
1 ControlPoint.draw();
```

Canvas.renderScene() ne ControlPoint naam explicitly nahi likha. Isse method flexible ban gaya.

Kisi doosre compatible object ke saath bhi use kiya ja sakta hai:

```
1 Canvas.renderScene.call(Guideline);
```

Agar Guideline ke paas draw() aur ctx available hain, to same renderScene() uske liye bhi work karega.

25. Loose coupling ka benefit

Hard-coded version hota:

```
1 var Canvas = {  
2     renderScene() {  
3         ControlPoint.draw();  
4     },  
5 };
```

Problem:

- Canvas directly ControlPoint par dependent ho gaya

- GuideLine ya kisi aur entity ke saath reuse mushkil hoga

Delegation version:

```
1 renderScene() {  
2     this.draw();  
3 }
```

Ab requirement sirf itni hai:

```
1 Jo bhi this context mile:  
2 1. uske paas ctx hona chahiye  
3 2. uske paas draw() hona chahiye
```

Canvas ko object ka exact type/name pata hona zaroori nahi.

Ye interface-like contract hai, though formal interface syntax use nahi hui.

26. Rotation flow

ControlPoint.rotate():

```
1 rotate(angleRadians) {  
2     var rotatedX =  
3         this.x * Math.cos(angleRadians) -  
4         this.y * Math.sin(angleRadians);  
5  
6     var rotatedY =  
7         this.x * Math.sin(angleRadians) +  
8         this.y * Math.cos(angleRadians);  
9  
10    this.setXY(rotatedX, rotatedY);  
11 }
```

For initial point:

```
1 x = 30  
2 y = 40  
3 angle =  $\pi/2$ 
```

Values:

```
1  $\cos(\pi/2) \approx 0$   
2  $\sin(\pi/2) = 1$ 
```

Therefore:

```
1 rotatedX =  $30 \times 0 - 40 \times 1 = -40$ 
```

```
2 rotatedY = 30 × 1 + 40 × 0 = 30
```

Result:

```
1 (30,40) → (-40,30)
```

Then:

```
1 this.setXY(-40, 30);
```

- coordinates update karta hai
 - setXY() this.render() call karta hai
 - canvas clear hota hai
 - rotated point redraw hota hai
-

27. Guideline example ka purpose

Next example dikhata hai ki same Canvas behaviors ko ek completely different object ke saath reuse kiya ja sakta hai.

GuideLine ke paas:

```
1 var Guideline = {  
2     ctx: /* canvas context */,  
3  
4     setAnchors(sx, sy, ex, ey) {},  
5     draw() {},  
6     render() {},  
7 };
```

ControlPoint ek point draw karta tha. Guideline do points ke beech line draw karta hai.

Lekin dono same operations reuse karte hain:

```
1 Canvas.setOrigin.call(...)  
2 Canvas.renderScene.call(...)
```

Canvas ko fark nahi padta context:

- ControlPoint hai
- Guideline hai
- ya future ka koi aur drawable object

Bas compatible properties aur methods available hone chahiye.

28. lineAnchor() factory

```
1 function lineAnchor(x, y) {
2     var anchor = {
3         __proto__: Coordinates,
4         render() {},
5     };
6
7     anchor.setXY(x, y);
8     return anchor;
9 }
```

Yahaan anchor:

- Coordinates se linked hai
- isliye setXY(), setX() and setY() delegate kar sakta hai
- empty render() define karta hai

Empty render() kyun?

Because Coordinates.setXY() ke end mein:

```
1 this.render();
```

call hota hai.

Agar anchor ke paas render() nahi hota, to:

```
1 anchor.setXY(x, y);
```

ke time this.render() fail ho jaata.

Isliye no-op function diya gaya:

```
1 render() {}
```

Ye function intentionally kuch nahi karta, lekin expected behavioral contract satisfy karta hai.

29. GuideLine.setAnchors() flow

```
1 setAnchors(sx, sy, ex, ey) {
2     this.start = lineAnchor(sx, sy);
3     this.end = lineAnchor(ex, ey);
4     this.render();
5 }
```

Example:

```
1 GuideLine.setAnchors(-30, 65, 45, -17);
```

Creates:

```
1 GuideLine.start = { x: -30, y: 65 }  
2 GuideLine.end   = { x: 45,  y: -17 }
```

Then:

```
1 this.render();
```

means:

```
1 GuideLine.render();
```

render():

```
1 Canvas.renderScene.call(this);
```

Yahaan this === GuideLine, so:

```
1 Canvas.renderScene.call(GuideLine);
```

Canvas.renderScene() ke andar:

```
1 this.draw();
```

means:

```
1 GuideLine.draw();
```

GuideLine.draw():

```
1 Canvas.line.call(this, this.start, this.end);
```

means:

```
1 Canvas ka line behavior  
2 GuideLine ka ctx/start/end context
```

Eventually line draw hoti hai.

30. Dynamic linkage aur testing

Chapter ke according delegation testing mein flexible ho sakti hai.

Example:

```
1 Object.setPrototypeOf(someObject, MockCoordinates);
```

Production object ko temporarily mock delegate se link kiya ja sakta hai.

Ya:

```
1 var MockCanvas = {  
2   renderScene() {
```

```
3      // real drawing nahi
4      // sirf test information collect karo
5  },
6  };
```

Then method ko mock canvas par explicitly delegate kar sakte ho.

Conceptual example:

```
1  ControlPoint.render = function render() {
2      MockCanvas.renderScene.call(this);
3  };
```

Isse tests mein actual DOM canvas use kiye bina behavior observe kiya ja sakta hai.

Chapter ka argument ye hai ki dynamic prototype linkage aur dynamic this contexts code ko runtime par flexible bana sakte hain.

31. “Why this?” ka main question

Author khud question raise karta hai:

Context ko this se implicitly share kyun karein? Explicit parameter ki tarah pass kyun na karein?

Delegation version:

```
1  Canvas.pixel.call(this, this.x, this.y);
```

Explicit-context-data version:

```
1  Canvas.pixel(this.ctx, this.x, this.y);
```

Delegation mein context this ke through travel karta hai. Alternative style mein required dependencies arguments ke through pass hoti hain.

32. Without delegation version

Without prototype/context delegation:

```
1  var Canvas = {
2      setOrigin(ctx, x, y) {
3          ctx.translate(x, y);
4          ctx.scale(1, -1);
5      },
```

```

6
7     pixel(ctx, x, y) {
8         ctx.fillRect(x, y, 1, 1);
9     },
10
11     renderScene(ctx, entity) {
12         // clear canvas...
13         entity.draw();
14     },
15 };

```

Ab functions ko required values explicitly deni pad rahi hain:

```

1 Canvas.setOrigin(ControlPoint.ctx, 100, 100);
2 Canvas.pixel(this.ctx, this.x, this.y);
3 Canvas.renderScene(this.ctx, this);

```

Coordinates mein bhi entity explicitly pass ho rahi hai:

```

1 setX(entity, x) {
2     entity.x = Math.round(x);
3 }

```

Delegation version mein:

```

1 setX(x) {
2     this.x = Math.round(x);
3 }

```

Difference:

```

1 Explicit version:
2 setX(entity, x)
3
4 Delegation version:
5 setX(x)
6 this implicitly entity provide karta hai

```

33. Explicit context passing ka “tax”

Author ke according explicit parameter approach ka cost hai:

- function signatures longer ho jaate hain
- call-sites cluttered ho jaate hain
- har jagah ctx, entity, ya state pass karna padta hai
- methods ko wrapper functions ki zarurat pad sakti hai

Example:

```
1 setXY(entity, x, y)
```

versus:

```
1 setXY(x, y)
```

And:

```
1 Canvas.renderScene(this.ctx, this);
```

versus:

```
1 Canvas.renderScene.call(this);
```

Author delegation style prefer karta hai because implicit context sharing repetition reduce karti hai.

Lekin author acknowledge bhi karta hai ki kuch developers explicit style prefer kar sakte hain, kyunki dependencies clearly visible hoti hain.

34. Delegation versus explicit parameters

Dono approaches ke conceptual trade-offs:

Delegation with this

```
1 Coordinates.setXY.call(point, 3, 4);
```

Benefit

- fewer explicit parameters
- runtime composition
- methods flexible context par operate kar sakte hain
- prototype se convenient implicit delegation

Cost

- this ko correctly track karna padta hai
 - call-site bahut important ho jaata hai
 - debugging beginners ke liye confusing ho sakti hai
 - object ke required properties/methods hidden contract ban sakte hain
-

Explicit parameter passing

```
1 Coordinates.setXY(point, 3, 4);
```

Benefit

- dependency clearly visible
- function ko kis object par operate karna hai, argument se obvious hai
- this binding rules ka confusion kam

Cost

- signatures longer
- context repeatedly pass karna padta hai
- call-sites verbose ho sakte hain

Chapter neutral comparison briefly karta hai, lekin author clearly delegation style ko prefer karta hai.

35. this ko samajhne ka core rule

Is chapter ke almost saare examples understand karne ke liye ek rule yaad rakho:

this function kahaan defined hai usse determine nahi hota. Normal function ke case mein this generally function ko kaise call kiya gaya hai, usse determine hota hai.

Example:

```
1  var obj = {  
2      show() {  
3          console.log(this);  
4      },  
5  };  

```

Implicit call

```
1  obj.show();  
1  this = obj  

```

Explicit call

```
1  obj.show.call(other);  
1  this = other  

```

Same function, different context.

Delegation isi dynamic context ko code reuse ke liye leverage karti hai.

36. Ek compact working example

Chapter ke concept ko ek smaller working example se dekho:

```
1  var Coordinates = {
2      setX(x) {
3          this.x = x;
4      },
5
6      setY(y) {
7          this.y = y;
8      },
9
10     setXY(x, y) {
11         this.setX(x);
12         this.setY(y);
13     },
14 };
15
16 var Inspect = {
17     toString() {
18         return `(${this.x},${this.y})`;
19     },
20 };
21
22 var point = Object.create(Coordinates);
23
24 point.setXY(3, 4);
25
26 console.log(
27     Inspect.toString.call(point)
28 );
29 // "(3,4)"
```

Isme:

```
1  point.setXY(3, 4);
```

implicit delegation hai.

```
1  Inspect.toString.call(point);
```

explicit delegation hai.

Actual data:

```
1 point.x = 3
2 point.y = 4
```

Methods:

```
1 Coordinates → coordinate methods
2 Inspect      → display method
```

Runtime context:

```
1 this = point
```

Isliye different behavior objects same point par operate kar pa rahe hain.

37. Delegation, inheritance aur composition ka difference

Inheritance

Thinking:

```
1 ControlPoint is a Point
```

Behavior class hierarchy se milta hai.

```
1 Point
2   ↓
3 ControlPoint
```

Traditional composition

Thinking:

```
1 ControlPoint contains/combines coordinate and rendering
  behavior
```

Methods/modules ko combine ya hold kiya ja sakta hai.

Delegation

Thinking:

```
1 ControlPoint coordinate work Coordinates se karwa sakta hai
2 ControlPoint drawing work Canvas se karwa sakta hai
```

Behavior ek object se another object ke shared context par execute hota hai.

```
1 ControlPoint → delegates to Coordinates
2 ControlPoint → delegates to Canvas
```

Chapter ka main argument hai ki JS ka prototype system naturally delegation model provide karta hai.

38. Important hidden contracts

Delegation examples flexible hain, lekin methods kuch assumptions bhi banate hain.

For example:

```
1 Canvas.renderScene.call(someObject);
```

tabhi work karega agar someObject ke paas:

```
1 ctx
2 draw()
```

available hon.

Similarly:

```
1 Coordinates.setXY.call(someObject, 3, 4);
```

given current implementation tabhi work karega agar someObject:

- setX() resolve kar sake
- setY() resolve kar sake
- render() resolve kar sake

Ye requirements function signature mein visible nahi hain. Ye implicit contract hain.

Isliye delegation powerful hai, lekin carefully designed object capabilities aur method expectations samajhni padti hain.

39. Chapter ka exact central argument

Author ye nahi keh raha ki:

- classes kabhi use mat karo
- delegation mainstream standard architecture hai
- major frameworks isi exact OLOO style mein likhe jaate hain

Actually preamble mein clear warning hai ki ye approach mainstream nahi hai.

Author ka deeper purpose hai:

1. class ke niche chalne wale JS mechanics ko samjhana
2. new, constructor, prototype aur this ko separate pieces ki tarah dekhna

3. dikhana ki same mechanics ko different design pattern mein combine kiya ja sakta hai
4. inheritance ke alternative ke roop mein delegation samjhana
5. JavaScript ke prototypal nature ko directly explore karna

Even if tum practical projects mein classes use karo, delegation understand karne se:

- method lookup better samajh aayega
 - prototype chain clearer hogi
 - this context clearer hoga
 - class syntax ke behind mechanics samajh aayenge
-

40. Final mental model

Poore chapter ko is mental model mein compress kar sakte ho:

Step 1: Behavior objects banao

```
1  var Coordinates = {
2      setXY(x, y) {
3          this.x = x;
4          this.y = y;
5      },
6  };
7
8  var Inspect = {
9      toString() {
10         return `${this.x},${this.y}`;
11     },
12  };
```

Step 2: Actual state object banao

```
1  var point = {};
```

Step 3: Behavior ko state object par delegate karo

```
1  Coordinates.setXY.call(point, 3, 4);
```

Step 4: Kisi aur behavior object ko same context do

```
1  Inspect.toString.call(point);
```

Result:

- | |
|-------------------------------|
| 1 Behavior Coordinates par |
| 2 Behavior Inspect par |
| 3 Data point par |
| 4 Shared context this = point |

Yahi **virtual composition** hai.

Chapter ka short summary

- Constructor object create nahi karta, mainly initialize karta hai.
- new object creation, prototype linkage, function invocation aur return handling karta hai.
- Factory function manually object create aur return kar sakta hai.
- Factory function ko new ke saath use karna duplicate object creation cause kar sakta hai.
- Object.create(obj) new object create karke directly obj se prototype-link karta hai.
- Delegated method prototype par mil sakta hai, lekin this calling object hota hai.
- .call(obj) ke through explicit delegation hoti hai.
- Prototype method call ke through implicit delegation ho sakti hai.
- Different behavior objects same this context par operate karke runtime composition produce karte hain.
- Author is runtime behavior combination ko **virtual composition** kehta hai.
- **OLOO** ka meaning hai "Objects Linked to Other Objects."
- Canvas, Coordinates, ControlPoint, aur GuideLine parent-child classes nahi, cooperating peer objects hain.
- this context ko implicitly carry karta hai; alternative mein same context explicit function parameters se pass ki ja sakti hai.
- Delegation flexible hai, lekin this, prototype resolution aur hidden behavioral contracts ki strong understanding maangti hai.

Explanation approach

Maine chapter ka same progression follow kiya: **factory → prototype linkage → Object.create() → explicit/implicit delegation → virtual composition → Canvas example → explicit context alternative**. Saath hi input ke do important technical issues highlight kiye: Object.create() passed object se directly link karta hai, aur initial point explicit-delegation example as written this.setX() par fail ho sakta hai.